

LLM-Guided Performance Engineering for GPU Kernels

Данил Сивцов, Булат Шарипов, Тимур Пономарев

2 августа 2026 г.

Аннотация

В рамках данной работы исследуется, может ли LLM-агент без дообучения писать и оптимизировать FP8 и FP4-ядра на CuTe DSL для NVIDIA — фреймворке, слабо представленном в обучающих данных моделей. Для этого были исследованы методы создания харнесса, который предоставит агенту документацию и примеры, исполняет кандидаты и возвращает ошибки компиляции, численные расхождения и данные профилировщика.

На пяти одиночных операциях агент без справки получил 0/30 успешных решений, а добавление примеров и разбора частых ошибок подняло результат до 28/30. На полноценных блоках GPT-2 и Qwen лучшие корректные реализации достигли ускорения $11.35\times$ и $79.67\times$ относительно PyTorch-бейзлайна. Task-aware contrast-контекст сократил расход входных токенов в $8.0\times$ без заметного снижения корректности: 17/24 против 16/24, $p = 1.0$. При этом динамический RAG не дал стабильного прироста и на Gemma ухудшил результат. Эксперименты показывают, что точный контекст и полная обратная связь важнее объёма документации, однако перенос на составные out-of-sample слои всё ещё ограничен способностями самой модели.

1 Введение

Скорость модели зависит от того, какую долю пиковой производительности ускорителя удаётся использовать на практике. Одно ядро почти непрерывно загружает вычислительные блоки полезной работой и приближается к пиковым FLOPS, другое заставляет их значительную часть времени ждать данные — и достигает лишь небольшой доли доступной производительности. По этой причине две реализации одной и той же операции могут отличаться по скорости в разы. Следовательно, один из основных способов ускорить LLM — переписать её ядра эффективнее. Проблема в том, что это тяжело и долго: нужен инженер, знающий и модель, и архитектуру конкретной видеокарты. Как следствие, большие ядра, где целый слой трансформера считается одним куском без промежуточных записей в память, почти никто не пишет вручную. Именно там и остаётся неиспользованный запас скорости.

Отсюда цель проекта: агент, который пишет и оптимизирует ядра сам. Препятствие в том, что небольшие модели плохо владеют двумя вещами сразу. CuTe DSL — фреймворк NVIDIA для написания ядер; модели уверенно пишут на более распространённом Triton, но по CuTe у них мало обучающих данных. FP8 — восьмибитный числовой формат; он даёт скорость, но требует явно управлять масштабами и округлениями, часть вычислений идёт в FP32, а результат должен быть округлён до FP8, иначе ответ не совпадёт с эталоном. Вклад работы состоит из трёх частей.

1. Показано влияние различных способов добавления контекста и инструкций для задачи генерации ядер на фреймворке и типе данных, на которых модель не была обучена.
2. На основе проведённых экспериментов был разработан и предложен программный харнесс, позволяющий успешно решать пять типовых задач с использованием CuTe и формата данных FP8. Разработанное решение сделало эти задачи доступными для модели DeepSeek-v4-flash и обеспечило ускорение в среднем до 9 раз по сравнению с базовой реализацией в формате BF16.
3. Продемонстрировано, что компактное представление контекста и адаптивный роутинг через RAG позволяет сократить расход токенов в 8 раз без существенной потери качества

2 Исследовательские вопросы

В рамках работы мы рассматриваем подходы к построению эффективной обвязки (харнесса) вокруг модели, и следовательно, ставим следующие исследовательские вопросы:

- Q1. Какую документацию подавать, чтобы агент освоил незнакомый фреймворк? (§4)
- Q2. Как её доставлять: отдать файлами или вложить в промпт целиком? (§5)
- Q3. Какая обратная связь между итерациями помогает: второй агент-рецензент? (§5)
- Q4. Помогает ли профилировщик и в каком виде подавать его данные? (§5, §6)
- Q5. Можно ли всем этим ускорить настоящий слой настоящей модели? (§6)
- Q6. Можно ли сжать документацию под задачу без потери корректности? (§7)
- Q7. Помогает ли выбор подсказок по текущей ошибке поверх компактной документации? (§7)
- Q8. Переносится ли такая специализация на незнакомый составной слой и более слабые модели? (§7)

3 Постановка

Модель-агент. Сначала мы подняли собственные Gemma и Qwen3.6-35B-A3B на $2\times V100$: хотелось, чтобы ядра писала полностью локальная модель. Они оказались слишком слабы, до рабочего ядра не доходило почти никогда (Более подробно в параграфах 7 и 8). Поэтому перешли на DeepSeek v4 Flash — модель умнее, но всё ещё небольшая, что и требовалось: если обвязка вытягивает её, значит работает именно обвязка, а не эрудиция фронтальной модели.

Задачи и инфраструктура. За основу взят KernelBench — открытый бенчмарк, где модели дают эталонную реализацию слоя на PyTorch и просят написать более быстрое GPU-ядро; ответ засчитывается при численном совпадении с эталоном, а метрика — ускорение относительно него. Задач под CuTe и FP8 не существует, поэтому мы взяли формат KernelBench и написали свои: собственные эталоны, входы и эпилоги в FP8-семантике, свои пороги расхождения. Первый уровень — одиночная операция, второй — объединенная последовательность (GEMM вместе с эпилогом); эксперименты E1–E4 прогонялись на пяти таких задачах, по две первого уровня и три второго. Для оценки мы использовали серверы с NVIDIA B300: агент отправляет туда код, там он компилируется, сверяется с эталоном, замеряется и профилируется. Эталон перезамеряется внутри каждого запуска, чтобы загрузка машины не искажала сравнение.

Фреймворк агента. Для быстрого старта агент был просто opencode в headless-режиме. Затем мы перенесли всё в целевой фреймворк KernelEvo: взяли ветку с агентским поведением и поверх неё интегрировали оценщик B300 и наши задачи. Агент правит только свой единственный файл с ядром. Проверка корректности и эталонная реализация ему недоступны, подменить их он не может.

План экспериментов. Каждый эксперимент меняет ровно один параметр относительно общей опорной конфигурации, поэтому все сравнения однопараметрические: E1 — уровни документации (Q1), E2 — способ доставки того же материала (Q2), E3 — агент-рецензент (Q3), E4 — сводка профилировщика (Q4). Затем лучшее переносится на настоящие слои (Q5). Отдельно мы сравниваем полный и специализированный контекст (Q6), проверяем динамический выбор подсказок по ошибке (Q7) и перенос специализации на out-of-sample составной блок (Q8). Единица измерения — ход: агент пишет ядро, отправляет на GPU, получает диагностику, переходит к следующему ходу. В таблицах «PASS k/n » означает, что корректное ядро получено в k ходах из n ; «лучшее» — время быстрого корректного ядра; «ускорение» — во сколько раз оно быстрее эталона.

4 E1: какую документацию подавать

Документация разбита на четыре кумулятивных уровня, не зависящих от задачи: только задача → плюс документация (описание API, алгебра layout’ов, архитектура Blackwell, численные свойства FP8) → плюс примеры (нейтральные фрагменты рабочего кода) → плюс разбор ошибок. Каждый уровень включает предыдущий целиком, поэтому разница между соседними строками — ровно вклад добавленного материала.

Таблица 1: **E1: какая документация позволяет агенту решать задачи.** Четыре кумулятивных уровня документации, каждый включает предыдущий целиком; на каждом уровне агент получает по шесть ходов на каждой из пяти задач, всего 30 ходов. PASS — в скольких ходах получилось численно корректное ядро; ускорение — насколько лучше ядро быстрее PyTorch-эталона своей задачи (между задачами несопоставимо, поэтому итог подводится по PASS). **Одних знаний из предобучения модели не хватает:** без справки она не решает ничего. Обычного описания API тоже мало — перелом наступает, когда агенту дают конкретные примеры кода, а разбор частых ошибок делает результат ещё стабильнее и заметно дешевле.

Задача	Только задача		+ документация		+ примеры		+ разбор ошибок	
	PASS	Ускор.	PASS	Ускор.	PASS	Ускор.	PASS	Ускор.
l1_02, масштабирование FP4	0/6	—	4/6	2.35×	6/6	3.15×	6/6	3.31×
l1_40, LayerNorm	0/6	—	6/6	74.77×	5/6	23.68×	5/6	40.34×
l2_12, GEMM + LeakyReLU	0/6	—	3/6	0.05×	5/6	1.87×	5/6	2.75×
l2_14, GEMM + деление	0/6	—	0/6	—	6/6	3.22×	6/6	3.10×
l2_63, GEMM + ReLU	0/6	—	0/6	—	5/6	3.23×	6/6	3.29×
Итого	0/30		13/30		27/30		28/30	
Входные токены	—		3.88 М		1.79 М		1.19 М	
Выходные токены / рассуждения	343 646 / 1.44 М		399 607 / 1.52 М		386 726 / 1.26 М		334 784 / 1.12 М	

Без справки агент не решает ничего ни на одной из пяти задач: 0 из 30 ходов. Документация переводит через порог лишь частично, 13 из 30, и распределены эти решения неравномерно: на трёх более простых задачах она работает, а на обеих слитых операциях уровня 2 даёт ровно ноль, причём ошибки там характерны — на l2_63 из 21 проверки не прошла ни одна, и восемь ошибок приписались на обращения к функциям, которых в библиотеке нет. Примеры поднимают результат до 27 из 30 и работают уже на всех пяти задачах, разбор ошибок закрепляет, 28 из 30.

Стоимость меняется монотонно в ту же сторону, причём по всем трём статьям сразу. Уровень документации — самый дорогой не только по входу (3.88 млн токенов на решение против 1.19 млн у лучшего уровня), но и по выходу, и по рассуждениям: 399 607 и 1.52 млн против 334 784 и 1.12 млн. На документации модель пишет больше всех, думает больше всех и решает меньше всех из трёх работающих уровней.

Инсайт. Модель совсем не умеет решать задачи, используя только знания из своего предобучения. При этом, обычной API документации ей не хватает — можно заметить хороший уровень решения, предоставив ей конкретные примеры. Разбор частых ошибок помогает тратить меньше ресурсов и стабильнее получать результаты

5 E2–E4: доставка, рецензент, профилировщик

Опора для всех трёх — лучший уровень E1. В E2 тот же материал вкладывается в промпт целиком вместо того, чтобы лежать файлами, которые агент открывает сам. В E3 между ходами работает второй агент-рецензент:

он видит ядро и диагностику, но ничего не исполняет и не правит, только советует, а его токены учитываются отдельно. В Е4 следующему ходу возвращается сводка профилировщика с eval-сервера.

Таблица 2: **Е2–Е4: три способа помочь агенту** Бейзлайн — лучший уровень Е1 (разбор ошибок, документация файлами по требованию). Е2 вкладывает тот же материал целиком в промпт; Е3 добавляет между ходами агента-рецензента, который читает код и диагностику, но ничего не исполняет; Е4 возвращает следующему ходу сводку профилировщика. Колонки как в табл. 1. **Корректность не выросла ни от одного:** промпт и профилировщик нейтральны (27/30 и 28/30 против 28/30), а рецензент — единственный устойчиво отрицательный эффект.

Задача	Бейзлайн		Е2: промпт		Е3: рецензент		Е4: профилировщик	
	PASS	Ускор.	PASS	Ускор.	PASS	Ускор.	PASS	Ускор.
l1_02, масштабирование FP4	6/6	3.31×	6/6	3.25×	4/6	3.13×	5/6	2.92×
l1_40, LayerNorm	5/6	40.34×	5/6	68.50×	5/6	70.42×	6/6	16.37×
l2_12, GEMM + LeakyReLU	5/6	2.75×	6/6	2.91×	6/6	1.36×	6/6	1.39×
l2_14, GEMM + деление	6/6	3.10×	6/6	3.16×	5/6	1.36×	6/6	3.15×
l2_63, GEMM + ReLU	6/6	3.29×	4/6	3.54×	4/6	3.29×	5/6	3.38×
Итого	28/30		27/30		24/30		28/30	
Вход на решение	1.19 M		1.67 M		1.74 M		1.21 M	
Выход / рассуждения	334 784 / 1.12 M		341 663 / 1.34 M		340 742 / 1.37 M		307 939 / 1.02 M	

Е2: доставка (Q2). По устойчивости разницы почти нет, 27 из 30 против 28 из 30, то есть один ход. Зато вставка в промпт поднимает потолок оптимизации: лучшее ускорение выше на четырёх задачах из пяти. Платой становятся сбои на самых длинных задачах — только у этой конфигурации есть таймауты, три хода из шести на l2_63 не дошли до проверки. Материал всегда перед глазами, поэтому обращений к несуществующему API нет ни одного, но он же вытесняет рабочий контекст: модель знает, что писать, но теряет нить того, что уже написала.

Инсайт. Доставка файлами по требованию — разумный режим по умолчанию: в 1.4 раза дешевле на решение при той же устойчивости. Вставка в промпт систематически поднимает потолок оптимизации, поэтому уместна на доводке уже работающего ядра. На крупных задачах выбор меняется: там агент открывает бандл по одному файлу, каждый результат инструмента переносится в контекст всех следующих шагов, и один ход целиком уходил на чтение (52 обращения, 40 шагов) без единой строчки ядра. Поэтому в §6 используется промпт.

Е3: рецензент (Q3). Единственная конфигурация, которая заметно проигрывает опоре: 24 из 30, хуже на трёх задачах из пяти, и сверх того 378 951 токен самого рецензента. Диагноз виден в характере ошибок: на l2_63 из 11 неудачных проверок десять — это обращения к несуществующему API. Рецензент воспроизводит ровно ту патологию, которую мы видели в Е1: он читает документацию, но ничего не исполняет, поэтому его советы правдоподобны по форме и называют несуществующие функции. Основной агент доверяет им больше, чем компилятору: совет приходит как внешний авторитет, а сообщение об ошибке нужно ещё интерпретировать.

Инсайт. Советчик, который ничего не исполняет, — дополнительный источник галлюцинаций, а не фильтр от них, и это единственный отрицательный эффект, воспроизводящийся между задачами. Если возвращаться к идее рецензента, у него должно быть право самому запустить компиляцию и высказываться только о проверенных гипотезах.

Е4: профилировщик (Q4). Профилировать можно только работающее ядро, поэтому сводка появляется лишь после первого успеха и по построению не может помочь пройти тест впервые. Так и вышло: по устойчивости ровно ничья, 28 из 30 против 28 из 30, при почти той же цене решения. По ускорению эффект неотличим от шума и разнонаправлен — лучше на двух задачах, хуже на трёх. Интереснее траектория внутри запуска: на l2_63 конфигурация с профилировщиком до пятого хода отстаёт (0.2999, ошибка, 0.3050, 0.1515, 0.1269) и обгоняет ровно на последнем, шестом ходу (0.1133), тогда как опора выходит на плато уже с третьего хода, с 0.1261 до 0.1160, и почти не улучшается.

Инсайт. Профиль — отложенная инвестиция: сначала нужно любое работающее ядро, потом осмыслить его разбор, и только тогда начинается целенаправленная оптимизация вместо угадывания. На шести ходах она не окупается ни на одной задаче, но кривая на последнем ходу всё ещё падает на 11 % за ход, значит проверять надо удлинением горизонта, а не повтором той же конфигурации.

6 Проверка на настоящем слое

Задачи Е1–Е4 — одна слитая операция, эталон исполняется за доли миллисекунды. Вопрос Q5: переносится ли найденное на целый слой реальной модели. Мы добавили две задачи, где агент реализует весь блок трансформера одним набором CuTe-ядер: блок GPT-2 Small (эталон 3.594 мс) и блок Qwen3.5-4B (эталон 49.885 мс, в 15 раз больше по весам).

Конфигурация — лучшая из найденных в Е1–Е4: уровень «разбор ошибок», доставка в промпт. Горизонт увеличен с шести ходов до девяти, потому что Е4 показал, что шести профилей мало. Заодно сравниваются два формата сводки: стандартный агрегирует время по именам ядер, а формат с таймлайном дополнительно даёт одну характерную итерацию замера — начало, конец и простой перед каждым запуском.

Таблица 3: **Оптимизация целого трансформерного блока** Использована лучшая конфигурация из E1–E4, девять ходов на прогон. Профилировщик выключен, либо стандартный (время по именам ядер), либо с таймлайном (плюс одна итерация замера: начало, конец и простой каждого запуска). PASS — корректных ходов из всех; лучшее и ускорение — относительно PyTorch-эталоны своего блока, 3.594 мс у GPT-2 и 49.885 мс у Qwen. Контроль без профилировщика на GPT-2 прогонялся раньше и на шести ходах, что видно по знаменателю PASS. **Оба настоящих слоя ускоряются, 11.35× и 79.67×, и весь код написан агентом.**

Задача	Профилировщик	PASS	Лучшее, мс	Ускор.	Входные токены	Выходные токены
GPT-2	выключен	4/6	0.5205	6.90×	6.83 М	61,553
GPT-2	стандартный	8/9	0.8295	4.33×	10.91 М	145,472
GPT-2	с таймлайном	8/9	0.3164	11.35×	12.42 М	137,511
Qwen	выключен	6/9	0.6262	79.67×	14.66 М	132,493
Qwen	стандартный	8/9	51.1809	0.97×	10.65 М	112,034
Qwen	с таймлайном	8/9	7.3564	6.78×	12.98 М	163,637

Оба слоя решаются: лучшие ядра — 11.35× на GPT-2 и 79.67× на Qwen над эталоном при 6–8 успешных ходах из девяти. Удлинение горизонта окупилось — в четырёх конфигурациях из пяти лучший результат пришёлся на восьмой или девятый ход, то есть на шести ходах мы бы его не увидели.

Таблица 4: **Как шёл прогресс по ходам.** Ячейка - время кандидата на каждом ходу, мс; «ошибка» — ход, не давший корректного ядра. **Видно, что таймлайн улучшался степенями по мере поступления сводок — в отличие от конфигурации без профиля, выигравшей одним скачком.**

Задача	Профилировщик	1	2	3	4	5	6	7	8	9
GPT-2	выключен	ошибка	ошибка	78.3	5.45	5.45	0.52	—	—	—
GPT-2	стандартный	ошибка	4.08	4.09	3.61	3.63	3.63	0.83	0.83	0.83
GPT-2	с таймлайном	4.04	ошибка	1.19	1.19	0.51	0.35	0.35	0.35	0.32
Qwen	выключен	ошибка	ошибка	160.1	5.85	5.85	ошибка	3.21	0.63	0.63
Qwen	стандартный	ошибка	279.9	51.2	51.2	51.2	51.2	51.2	51.2	73.5
Qwen	с таймлайном	ошибка	220.0	220.3	50.3	9.42	51.5	9.41	9.42	7.36

Формат с таймлайном обошёл стандартный на обеих задачах: 11.35× против 4.33× на GPT-2 и 6.78× против 0.97× на Qwen, где стандартный ни разу не обогнал обычный эталон.

Различаются и формы прогресса, они видны в таблице 4. С таймлайном лучшее время падало степенями по мере поступления сводок — 220 → 50.3 → 9.4 → 7.4 мс на Qwen. Конфигурация без профиля выиграла там иначе, одним удачным ходом с 3.21 до 0.63 мс, простояв до этого шесть ходов на 5.85 мс.

Инсайт. Из двух форматов профиля рабочим оказался таймлайн: он выиграл у стандартного на обеих задачах и давал плавное улучшение по мере поступления сводок, тогда как выигрыш конфигурации без профиля на Qwen пришёл одним случайным скачком после шести ходов на плато.

Инсайт. Ответ на Q5 положительный: обвязка выводит небольшую модель без дообучения на ускорение настоящего слоя. Полный блок GPT-2 доведён с 3.594 до 0.3164 мс, блок Qwen3.5-4B — с 49.885 до 0.6262 мс, и весь код написан агентом на незнакомом ему CuTe DSL в FP8, без единой строчки от человека. Это ровно тот случай, ради которого всё затевалось: слитые ядра на целый слой вручную почти никто не пишет, потому что дорого и долго, — и именно там оставался неиспользованный запас скорости.

7 Task-aware контекст, динамический RAG и перенос

В этой секции исследуется не состав документации вообще, а её выбор для конкретной задачи. Мы последовательно проверяем четыре утверждения: сохраняет ли компактный task-aware пакет корректность полного бандла; важно ли содержание пакета при фиксированном размере; помогает ли динамически добавлять материалы после ошибки; переносится ли специализация с отдельных примитивов на целый блок трансформера. Эксперименты используют разные протоколы, поэтому их абсолютные числа не объединяются в одну метрику.

7.1 Компактная документация против полного бандла (Q6)

Исследовательский вопрос. Полный набор материалов гарантирует, что нужный рецепт присутствует в контексте, но вместе с ним модель получает десятки тысяч токенов о других примитивах. **Мы проверяем, можно ли удалить эту нерелевантную часть без измеримого снижения корректности.** В плече *compact* остаются только рецепты текущей задачи. Плечо *full* является строгим надмножеством *compact*: оно содержит те же полезные файлы и добавляет около 26 тысяч токенов общей документации.

Протокол. Эксперимент проведён на Qwen3.6-35B-A3B и трёх задачах: масштабировании FP4, LayerNorm FP8 и ConvTranspose3D FP8. Для каждой пары задача-argm запущено восемь независимых цепочек длиной до шести ходов. Модель, starter-кандидат, лимит вывода и B300-оценщик одинаковы; цепочка останавливается после первого PASS. Цепочка считается решённой, если хотя бы один кандидат прошёл численную проверку не позже шестого хода. Input tokens суммируются до первого PASS, а для нерешённых цепочек — по всему израсходованному бюджету.

Таблица 5: Сравнение компактной документации(compact) и полной(full) на Qwen. Compact потребовал в 8.0× меньше входных токенов без обнаруженного снижения корректности: 17/24 решённых цепочек против 16/24. Для каждой задачи указано число решённых независимых цепочек из восьми после не более чем шести ходов. Success@ k — macro-average доли цепочек, получивших первый PASS не позже хода k ; входные токены суммируются до первого PASS либо до исчерпания бюджета. Full содержит весь compact и дополнительно общую документацию, поэтому сравнение изолирует эффект лишнего нерелевантного контекста.

Задача или метрика	Compact	full
Масштабирование FP4	7/8	8/8
LayerNorm FP8	8/8	8/8
ConvTranspose3D FP8	2/8	0/8
Решено цепочек	17/24	16/24
Success@1, macro	45.8%	54.2%
Success@6, macro	70.8%	66.7%
Входных токенов	273 052	2 191 414

Результат. Compact решил 17/24 цепочки, full — 16/24; двусторонний точный критерий Фишера не обнаруживает различия ($p = 1.0$). При этом полный бандл израсходовал в 8.0× больше входных токенов. Мы не утверждаем строгую эквивалентность двух режимов: доверительные интервалы при 24 цепочках широки. Защищаемый вывод состоит в том, что на данной выборке удаление нерелевантной документации не снизило наблюдаемую корректность и радикально уменьшило стоимость контекста.

Agentic follow-up. Отдельно мы проверили, переносится ли экономия токенов на поведение агента с файловыми инструментами. Использованы шесть задач, две независимые сессии на задачу и arm и три попытки мутации внутри сессии: 12 runs и 36 attempts на строку таблицы. *Solved run* означает, что хотя бы одна из трёх попыток дала PASS; *passing attempts* считает все корректные попытки. Помимо input tokens учитываются все tool calls и чтения файлов.

Таблица 6: Стоимость compact и full в agentic-пайплайне. Благодаря сжатию контекста(compact) сократился расход входных токенов на обеих моделях без статистически значимого снижения корректности; сокращение tool calls и чтений файлов наблюдалось только на Qwen. Каждая строка агрегирует шесть задач, по две независимые сессии на задачу и три mutation attempts на сессию: всего 12 runs и 36 attempts. Runs — сессии хотя бы с одним PASS; Attempts — все корректные кандидаты. Input, Tools и Reads — суммарные расходы по всем сессиям; arms следует сравнивать только внутри одной модели.

Модель	Контекст	Runs	Attempts	Вход, ток.	Tools	Reads
Qwen	Compact	7/12	19/36	12.47 M	947	595
Qwen	Full	6/12	10/36	36.57 M	2450	1721
Gemma	Compact	12/12	25/36	7.39 M	538	185
Gemma	Full	8/12	17/36	11.31 M	606	180

На Qwen compact уменьшил вход в 2.93×, tool calls в 2.59× и reads в 2.89×. На Gemma вход сократился в 1.53×, но число инструментальных действий почти не изменилось. Различия correctness не достигают уровня 0.05: для passing attempts $p = 0.0537$ на Qwen и $p = 0.0935$ на Gemma. Следовательно, токенная экономия воспроизводится на обеих моделях, а сокращение tool use является модельно-зависимым эффектом.

7.2 Релевантность при фиксированном размере контекста (Q6)

Исследовательский вопрос. Предыдущий эксперимент смешивает два свойства: compact короче и одновременно релевантнее полного бандла. Чтобы проверить именно маршрутизацию, мы сравнили правильный compact с нерелевантным текстом того же размера. Если модели помогает просто короткий префикс, два плеча должны показывать одинаковый результат.

Протокол. Использованы Qwen и три задачи: масштабирование FP4, LayerNorm FP8 и GEMM+LeakyReLU FP8. Во всех arms совпадают TASK.md, starter, модель, temperature 0.7, лимит вывода 4096 токенов, B300-оценщик и бюджет до шести ходов. На задачу и arm выполнено восемь независимых цепочек. *Matched-irrelevant* детерминированно собирается из материалов других разделов и не содержит файлов correct compact. Его статистический размер совпадает с correct compact с отклонением не более 0.1%; третий arm получает только условие задачи.

Результат. Correct compact решил 14/24 цепочки, тогда как matched-irrelevant и task-only не решили ни одной. Для агрегированного сравнения correct compact против matched-irrelevant двусторонний Fisher exact даёт $p = 8.13 \cdot 10^{-6}$. Следовательно, эффект нельзя объяснить только длиной промпта: важна релевантность выбранных recipes и examples. Ограничение видно из разбивки по задачам: все 14 решений приходятся на FP4 scale и LayerNorm, а сложный GEMM не решает ни один arm. Маршрутизация помогает на доступных модели задачах, но не выводит её из принципиально сложного режима отказа.

Таблица 7: Контроль релевантности при фиксированном размере контекста. При одинаковом размере промпта релевантный контекст решил 14/24 цепочки, тогда как нерелевантный контекст и task-only не решили ни одной. В каждой ячейке указано число решённых цепочек из восьми после не более чем шести ходов. Task-only не получает документацию; matched-irrelevant получает нерелевантный текст того же статического размера, что correct compact, с отклонением не более 0.1%; correct compact получает релевантные task-aware материалы. Поэтому сравнение двух последних столбцов изолирует содержание контекста от его длины.

Задача	Только задача	Matched-irrelevant	Correct compact
Масштабирование FP4	0/8	0/8	7/8
LayerNorm FP8	0/8	0/8	7/8
GEMM + LeakyReLU FP8	0/8	0/8	0/8
Итого	0/24	0/24	14/24

7.3 Error-conditioned RAG поверх compact (Q7)

Исследовательский вопрос. Статический compact выбирается до запуска и не реагирует на конкретную ошибку кандидата. Мы проверяем, помогает ли расширять его динамически: после неудачной внутренней проверки retriever классифицирует compiler, runtime или numerical diagnostic и добавляет две локальные карточки. Контроль использует тот же compact, но не получает дополнительных карточек.

Протокол. Расширенная матрица проведена на Qwen и Gemma по шести задачам. Для каждой модели и arm выполнены 12 runs — по две сессии на задачу — и три mutation attempts внутри run, всего 36 attempts. Starter, evaluator и исходный compact одинаковы. Основные метрики: solved runs, passing attempts, provider-reported input tokens, tool calls и число evaluations. Такое разделение важно: большее число обращений к оценщику может означать как полезный feedback loop, так и заикливание на повторяющейся ошибке.

Таблица 8: Корректность и стоимость error-conditioned RAG. RAG не дал воспроизводимого выигрыша: на Qwen рост оказался незначимым, а на Gemma корректность снизилась. Каждая строка агрегирует шесть задач, по две сессии на задачу и три attempts на сессию: 12 runs и 36 attempts. Compact является общим статическим контекстом; в RAG-arm после неудачной B300-проверки добавляются две карточки, выбранные по диагностике. Runs и Attempts определены как в табл. 6; Input, Tools и B300 — суммарные расходы. Arms сравниваются только внутри одной модели.

Модель	Arm	Runs	Attempts	Вход, ток.	Tools	B300
Qwen	Compact	6/12	18/36	13.40 M	999	64
Qwen	Compact + RAG	9/12	20/36	17.76 M	1185	80
Gemma	Compact	9/12	18/36	8.51 M	546	85
Gemma	Compact + RAG	2/12	5/36	8.16 M	628	125

Результат. На Qwen RAG увеличил solved runs с 6/12 до 9/12, но ни эта разница ($p = 0.4003$), ни 18/36 против 20/36 passing attempts ($p = 0.8136$) неотличимы от дисперсии. При этом input вырос в $1.33\times$, tools в $1.19\times$, а B300 evaluations в $1.25\times$. На Gemma эффект отрицателен: passing attempts упали с 18/36 до 5/36 ($p = 0.0020$), solved runs — с 9/12 до 2/12 ($p = 0.0123$), а evaluations выросли на 47%. Текущая реализация retrieval маршрутизирует тематически правильные карточки, но не гарантирует, что модель сможет применить их к уже выбранной архитектуре. Поэтому RAG не входит в финальную конфигурацию харнесса.

7.4 Перенос на out-of-sample блок GPT-2 (Q8)

Исследовательский вопрос. Compact-пакеты строились по отдельным примитивам. GPT-2 Small проверяет их композиционный перенос: задача объединяет два LayerNorm, четыре GEMM, каузальное внимание, GELU и два residual-соединения. Для успеха недостаточно знать каждый рецепт отдельно — требуется согласовать формы промежуточных буферов, масштабы FP8 и границы вычислений в FP32.

Протокол. Для Qwen и Gemma выполнено по три независимые persistent-сессии с обязательным self-routing документации через AGENTS.md и бюджетом до 20 evaluations. Засчитывается только автономный PASS. Ручная правка лучшего кандидата проводится после завершения эксперимента и используется исключительно для диагностики расстояния до решения. Помимо PASS учитываются фактические evaluations и provider-reported input/output tokens: они позволяют отличить неспособность исправить численную ошибку от сбоя edit-evaluate цикла.

Таблица 9: Перенос task-aware документации на out-of-sample блок GPT-2 Small. Ни одна модель не решила GPT-2 автономно: Gemma дошла до численной проверки, тогда как две из трёх сессий Qwen не дошли до eval-a. Для каждой модели выполнены три независимые persistent-сессии с бюджетом до 20 evaluations. PASS учитывает только автономно полученные корректные решения; B300 — фактическое число отправленных кандидатов, Input/Output — суммарные provider-reported токены. Последующая ручная правка кандидата Gemma в PASS не включена.

Модель	Сессий	PASS	Eval	Вход, ток.	Выход, ток.
Gemma	3	0/3	50	6 891 471	169 566
Qwen	3	0/3	5	17 449 878	594 051

Результат и диагностика. Ни одна модель не получила автономный PASS. Gemma регулярно доходила до численной проверки, но заикнулась на четырёх локальных ошибках: масштабе γ в LayerNorm, FP8-квантизации residual path, ширине MLP workspace и переставленных scales двух проекций. После ручного исправления только этих мест её лучший кандидат прошёл eval с первой отправки. Это не model solve, но показывает, что модель построила почти правильную общую декомпозицию.

Qwen сделал только пять evaluations, причём все в одной из трёх сессий; две остальные сессии потратили контекст, но не дошли до оценщика. Поэтому два значения 0/3 не являются эквивалентной capability-оценкой: результат Gemma отражает повторяющиеся численные ошибки, а результат Qwen дополнительно смешан с operational failure агентного цикла.

Инсайт. Статический task-aware context дал два согласованных результата: сопоставимую full корректность при меньшей стоимости и 14/24 против 0/24 при сравнении с нерелевантным контекстом того же размера. Динамический retrieval не улучшил этот режим, а out-of-sample блок показал его границу: релевантные локальные рецепты не заменяют способность модели согласовать всё составное решение.

8 Адаптация харнесса под маленькие модели

В ходе экспериментов было попробовано несколько видов моделей, большие (Deepseek-V4-Flash-284B) и модели поменьше (Qwen-3.6-35B-A3B, Gemma-4-31B). Получилось что на менее конкретном харнессе результаты на маленьких модельках получились хуже, чем на больших (что ожидаемо), но в таблицах данные результаты не приводятся, поскольку как без харнесса, так и с не подробным харнессом модель не могла решать задачи (0/30). Вместо увеличения объёма документации мы изменили её гранулярность и способ выбора. Исходный харнесс предоставлял одинаковый накопительный набор API-документации и примеров для всех задач, поэтому агент должен был самостоятельно найти нужный материал, выбрать архитектуру и адаптировать пример. В task-aware конфигурации каждой задаче автоматически сопоставляются два-три compile-verified рецепта: точные API-вызовы, численный контракт и алгоритмический scaffold для соответствующего семейства операций. Таким образом, объём контекста уменьшился, но доля непосредственно применимой информации выросла.

Инсайт. Task-aware специализация позволяет небольшим моделям перейти от неработоспособного CuTe-кода к адаптации корректных KernelBench-ядер. Однако она частично переносит алгоритмическую работу из модели в подготовленный scaffold и потому не гарантирует композиционный перенос на составной out-of-sample слой.

9 Ограничения

При интерпретации результатов мы учитываем четыре найденных дефекта экспериментальной установки. Файловая изоляция устранила утечку готового ядра между конфигурациями. Проверка наличия собственного ядра в профиле исключила ложные PASS из-за переиспользованной памяти. Числовая ошибка была отделена от ошибки компиляции, а простой профилировщик пересчитан только между пользовательскими ядрами.

Главное статистическое ограничение — небольшое число задач и репликаций. Поэтому выводы о pass rate и многократной экономии токенов достаточно устойчивы, но небольшие различия во времени и корректности нельзя считать доказанными. В частности, равенство 17/24 и 16/24 означает, что мы не обнаружили ухудшения от compact-контекста, а не доказали строгую эквивалентность двух режимов.

10 Заключение

Небольшая модель без дообучения способна писать оптимизированные CuTe-ядра в FP8 вплоть до целого блока трансформера, с ускорением $11.35\times$ и $79.67\times$ над эталоном.

По Q1-Q5: описательной API-документации недостаточно, однако исполняемые примеры и разбор ошибок поднимают корректность до 28/30; файлы по требованию дешевле полного промпта; рецензент без права исполнять код добавляет галлюцинации; профиль полезен только после получения корректного ядра; в настоящем трансформерном блоке харнесс добивается ускорения.

По Q6 мы не обнаружили статистически значимого снижения корректности от task-aware compact-контекста, при этом он сократил расход входных токенов в $8.0\times$. По Q7 error-conditioned RAG не дал устойчивого выигрыша и на Gemma ухудшил результат. По Q8 анализ кандидатов показывает частичный перенос отдельных CuTe-рецептов, но ни Gemma, ни Qwen не смогли автономно согласовать формы, масштабы и границы FP32 на составном GPT-2 блоке.

Дальше следует проверить compact-контекст на большем числе задач и моделей, автоматизировать маршрутизацию без правил по task-id и увеличить число независимых репликаций. К динамическому retrieval и многоостровной оптимизации имеет смысл возвращаться только после такого масштабирования.